

CO5 AT1 CLOUD:

R. Deepasri (192511309)

Constraint-Based Problem Solving – Short Exam Answers

1. HDFS Storage for Media Platform

Constraints: Availability > 99.9%, replication ≤ 3 , cost efficient.

Design:

Media Files → HDFS → DataNodes

- Use **HDFS replication factor = 3** to provide fault tolerance.
- Store replicas on **different DataNodes/racks** using rack awareness.
- Use **NameNode HA** with Active and Standby NameNodes.
- Use **erasure coding** for less frequently accessed media to reduce storage overhead.
- Use larger HDFS blocks to efficiently store large media files.
- Add DataNodes horizontally when storage increases.
- Monitor disk/node health using Hadoop monitoring tools.

Justification: Replication provides high availability, while erasure coding for cold data reduces storage cost without exceeding the replication constraint.

2. MapReduce Log Processing Within 20 Minutes

Design:

Log Files → HDFS → Map → Shuffle/Sort → Reduce → Output

- Divide log files into suitable **HDFS blocks**.
- Run multiple **Mapper tasks in parallel**.
- Each Mapper extracts important fields such as IP, timestamp, status and URL.
- Use a **Combiner** to reduce intermediate data.
- Partition keys among Reducers for balanced workload.
- Use an appropriate number of Reducers according to the limited cluster nodes.
- Store output in HDFS.

Optimization:

- Use **compressed input** where appropriate.
- Avoid unnecessary data transfer.
- Use efficient serialization and filtering.
- Increase parallelism within available nodes.
- Monitor task execution and eliminate data skew.

Result: Parallel Map and Reduce tasks allow the log-processing job to complete within the 20-minute window despite limited nodes.

3. YARN Scheduler Policy

Requirements: Fairness + ETL priority + node-failure recovery.

Design:

Use **Capacity Scheduler** with separate queues:

Queue	Priority	Purpose
ETL	High	Time-sensitive ETL jobs
	Reporting	Medium Business reports
ML	Normal	Machine-learning jobs

- Give the **ETL queue higher capacity and priority**.
- Reserve some cluster capacity for reporting and ML to maintain fairness.
- Use **resource limits** to prevent one job from consuming the entire cluster.
- Enable **YARN ApplicationMaster recovery**.
- Failed containers are automatically restarted.
- NodeManager failure causes affected tasks to be rescheduled on healthy nodes.

Result: ETL receives priority while other workloads continue to receive fair resources, and failures do not stop the complete workflow.

4. SAN, NAS & Operational Database Ingestion

Architecture:

SAN + NAS + Operational DB → NiFi/ETL → Kafka → HDFS/Data Lake → Backup

- **Apache NiFi** collects files from SAN/NAS and data from databases.
- Use **CDC (Change Data Capture)** for operational databases to capture only changed records.
- Use Kafka for reliable streaming and buffering.
- Store data in HDFS/data lake.
- Use **checksums and metadata** to detect duplicate files/records.
- Maintain source IDs and timestamps for data lineage.
- Use HDFS replication or backup storage for recovery.
- Schedule incremental ingestion rather than repeatedly copying complete datasets.

Result: The architecture provides reliable backup, minimizes duplicate data and efficiently integrates heterogeneous sources.

5. End-to-End Hadoop Ecosystem

Architecture:

Data Sources → Apache NiFi → Kafka → HDFS → YARN → Spark/MapReduce → Hive → BI

- **NiFi:** Securely collects and routes data from different sources.
- **Kafka:** Provides reliable, scalable data streaming.
- **HDFS:** Provides distributed storage.
- **YARN:** Manages cluster resources.
- **Spark/MapReduce:** Performs data processing.
- **Hive:** Provides SQL-based analysis.
- **Security:** Use **Kerberos authentication, TLS encryption and HDFS permissions**.
- **High availability:** Use **NameNode HA**, ResourceManager HA and replicated data.
- **ETL integration:** NiFi performs data extraction, transformation and loading.
- **Monitoring:** Track data flows, node health and processing failures.

Constraint satisfaction:

- **Secure data movement:** TLS + Kerberos + controlled access.
- **High availability:** HDFS replication + HA services.
- **NiFi/ETL integration:** NiFi handles ingestion and transformation.
- **Fault tolerance:** Replication, checkpoints and automatic task recovery.
- **Scalability:** Additional DataNodes and processing nodes can be added when workload grows.